# Vector Embeddings, Similarity Metrics, and Vector Database Engines

*Created by Dr. David Gomillion with assistance from AI*

**Executive Summary & Core Concepts**

Traditional databases excel at finding records through exact structured matches (e.g., WHERE customer_id = 4019 or WHERE status = 'active'). However, modern artificial intelligence (AI) and enterprise search applications require searching data based on **conceptual meaning and semantic similarity** rather than exact keyword overlap.

A user searching an engineering database for *"battery thermal runaways"* should retrieve records describing *"catastrophic cell temperature spikes"* even if the specific words "thermal runaway" never appear in the text. This capability is enabled by **vector embeddings** and **vector databases**.

This reading explores the mathematical foundation of dense vector embeddings, vector distance metrics (Cosine, Euclidean L2, Dot Product), vector indexing mechanics (k-NN vs. HNSW/IVF Approximate Nearest Neighbors), and the architectural trade-offs between integrated relational vector extensions and dedicated vector stores.

**Core Learning Objectives**

1. **Dense Vector Representations:** Contrast traditional sparse keyword representations (TF-IDF/BM25) with dense neural embeddings generated by deep learning models.
2. **Mathematical Similarity Metrics:** Compute and evaluate Cosine Similarity, Euclidean (L2) Distance, and Dot Product for high-dimensional vector comparison.
3. **Vector Indexing Mechanics:** Understand why exact k-Nearest Neighbor (k-NN) brute-force search fails at scale and how Approximate Nearest Neighbor (ANN) algorithms (HNSW, IVF) achieve sub-linear query latency.
4. **Vector Storage Architectures:** Compare integrated relational vector capabilities (MariaDB Vector, PostgreSQL pgvector) with specialized vector engines (Chroma, Pinecone, Qdrant).

**Part 1: Foundations of Vector Embeddings**

**1.1 From Keywords to Semantic Vectors**

Historically, text search relied on **sparse vector representations** like Term Frequency-Inverse Document Frequency (TF-IDF) or BM25. In a sparse vector model, a document is represented by a vocabulary-sized array where each dimension corresponds to a specific dictionary word.

While sparse models excel at exact keyword lookups (such as matching part numbers or error codes), they fail to capture context, synonyms, or linguistic nuances.

**Dense vector embeddings** solve this limitation by passing input data (text, images, or audio) through a trained deep neural network (such as BERT or Sentence-Transformers). The neural network translates the

semantic meaning of the input into a compact, fixed-size array of floating-point numbers (typically 384, 768, or 1536 dimensions).

*Table 1: Architectural comparison between sparse keyword representations and dense neural vector embeddings.*

| **Feature / Dimension** | **Sparse Embeddings (e.g., TF-IDF / BM25)** | **Dense Vector Embeddings (e.g., Sentence Transformers)** |
|---|---|---|
| **Vector Dimensionality** | High dimensional (10,000s to 100,000s, mostly zero values) | Compact fixed dimensions (256, 384, 768, 1536 float values) |
| **Semantic Understanding** | Exact keyword matching only; fails on synonyms and paraphrasing | Captures conceptual meaning, synonyms, and semantic context |
| **Storage & Memory Layout** | Inverted index posting lists | Continuous float arrays (requires vector indexing for fast search) |
| **Primary Use Case** | Precision exact-match search (e.g., SKU numbers, legal IDs) | Semantic search, Q&A systems, RAG grounding context |

**1.2 High-Dimensional Vector Spaces**

In a dense embedding space, every position corresponds to a latent semantic feature learned by the embedding model. Words, sentences, or document chunks with similar meanings are mapped to coordinates that sit close together in high-dimensional geometric space.

High-Dimensional Conceptual Mapping Example:

"Battery Cell Overheating" -----> [ 0.024, -0.183, 0.892, ..., 0.114 ]

"Thermal Spike in Module" -----> [ 0.021, -0.179, 0.885, ..., 0.109 ] (Very Close!)

"Customer Invoice Payment" -----> [-0.412, 0.651, -0.102, ..., -0.881 ] (Far Away!)

Because conceptually related chunks cluster together, finding relevant information shifts from text string matching to calculating **geometric distance** between vectors.

**Part 2: Vector Distance Metrics & Mathematical Mechanics**

To determine which stored document chunks are most semantically relevant to a user query, the system converts the query string into a query vector ($\mathbf{Q}$) using the same embedding model, then measures the distance between $\mathbf{Q}$ and candidate document vectors ($\mathbf{D}$).

**2.1 Common Vector Distance Metrics**

*Table 2: Comparison of mathematical vector distance and similarity metrics.*

| Metric Name | Mathematical Formula | Geometric Meaning | Best Use Case |
|---|---|---|---|
| **Cosine Similarity** | $\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\Vert{}\mathbf{A}\Vert{} \Vert{}\mathbf{B}\Vert{}}$ | Measures the angle $\theta$ between two vectors, ignoring magnitude | Text semantic search where document lengths vary |
| **Euclidean Distance (L2)** | $d(\mathbf{A}, \mathbf{B}) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$ | Measures straight-line distance between two vector endpoints | Fixed-length spatial or image feature vectors |
| **Dot Product (Inner Product)** | $\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^{n} A_i B_i$ | Multiplies magnitudes and cosine of the angle | Normalized vectors (where vector length $\Vert{}\mathbf{A}\Vert{} = 1$) |

**2.2 Cosine Similarity vs. Cosine Distance**

Cosine similarity produces a score ranging from **-1.0** (opposite meaning) to **1.0** (identical meaning):

- $\cos(\theta) = 1.0$: Vectors point in the exact same direction ($\theta = 0^\circ$).
- $\cos(\theta) = 0.0$: Vectors are orthogonal/unrelated ($\theta = 90^\circ$).
- $\cos(\theta) = -1.0$: Vectors point in opposite directions ($\theta = 180^\circ$).

Most vector database engines require a **distance metric** (where smaller numbers indicate higher similarity) rather than a similarity score. **Cosine Distance** is calculated as:

$$\text{Cosine Distance} = 1 - \text{Cosine Similarity}$$

When vectors are normalized to unit length ($\Vert{}\mathbf{A}\Vert{} = 1$), Cosine Similarity, Dot Product, and Euclidean Distance become mathematically equivalent in ranking order, allowing databases to use ultra-fast Dot Product operations.

**Part 3: Vector Indexing & Database Architectures**

**3.1 k-Nearest Neighbors (k-NN) vs. Approximate Nearest Neighbors (ANN)**

Performing an exact **k-Nearest Neighbor (k-NN)** search requires computing the cosine distance between the query vector and *every single stored vector* in the database (brute-force flat scan).

While flat scans guarantee $100\%$ search accuracy (recall), the query latency grows linearly—$O(N)$—with the number of records. Searching millions of 768-dimensional vectors using a flat scan takes seconds, which is unacceptable for interactive systems.

To achieve sub-20 millisecond search response times across millions of records, vector databases build **Approximate Nearest Neighbor (ANN)** indexes. ANN indexes trade a tiny fraction of search accuracy (e.g., $95\text{--}98\%$ recall) for orders of magnitude faster query execution ($O(\log N)$ latency).

### 3.2 Dominant ANN Indexing Algorithms

*Table 3: Comparison of major Approximate Nearest Neighbor (ANN) indexing structures.*

| Indexing Algorithm | Processing Mechanism | Operational Pros | Operational Cons |
|---|---|---|---|
| **Flat / Exact (k-NN)** | Brute-force distance check against all stored vectors | $100\%$ precision/recall; no index building step | Scales linearly $O(N)$; slow for large datasets |
| **HNSW (Hierarchical Navigable Small World)** | Multi-layer proximity graph allowing skip-list graph traversal | Extremely fast query latency ($O(\log N)$); high recall | Higher RAM consumption to store graph edges |
| **IVF (Inverted File Index)** | Partitions vector space into Voronoi cells using k-means clustering | Reduced memory footprint; fast coarse-centroid search | Slightly lower recall; requires re-clustering as data grows |

### 3.3 Integrated Extensions vs. Dedicated Vector Engines

When architecting an enterprise system like **Project Gigavolt**, data architects must choose between adding vector capabilities to an existing relational database or deploying a dedicated vector store.

*Table 4: Architectural comparison between integrated relational vector extensions and dedicated vector stores.*

| Architectural Approach | Representative Technologies | Key Advantages | Trade-offs & Limitations |
|---|---|---|---|
| **Integrated Relational Extensions** | MariaDB Vector, PostgreSQL (pgvector) | Single database instance; transactional ACID safety; hybrid SQL joins (WHERE model = 'GIGA' AND vector_distance < 0.2) | Competes with relational workload for RAM and CPU resources |

| | | | |
|---|---|---|---|
| **Dedicated Vector Stores** | Chroma, Pinecone, Qdrant, Milvus | Highly optimized for ANN graph indexes; horizontally scalable vector search | Requires separate ETL sync pipelines; eventual consistency; dual database management |

In multi-engine enterprise architectures, a hybrid pattern is often applied: storing master transactional attributes in 3NF relational tables, while indexing text chunk embeddings in dedicated or extension-backed vector stores linked by chunk_id foreign keys.

**Part 4: Practical Reference Script**

The following complete Python script demonstrates generating dense vector embeddings using sentence-transformers, computing cosine similarity metrics using NumPy, and executing both flat k-NN and top-$k$ similarity queries over unstructured enterprise text chunks.

```
# ============================================================================

# ISTM 622: Vector Embedding Generation & Similarity Search Reference Pipeline

# Focus: Sentence Embeddings, Cosine Similarity, and Top-K Vector Retrieval

# Language: Python 3.10+

# Dependencies: sentence-transformers, numpy

# ============================================================================


import numpy as np

from typing import List, Dict, Any

from sentence_transformers import SentenceTransformer



class VectorSearchEngine:

  def __init__(self, model_name: str = "all-MiniLM-L6-v2"):

    """Initializes the embedding model (384-dimensional dense vectors)."""

    print(f"Loading embedding model: {model_name}...")

    self.model = SentenceTransformer(model_name)

    self.vectors: np.ndarray = np.empty((0, 384))

    self.documents: List[Dict[str, Any]] = []
```

```
  def add_documents(self, docs: List[Dict[str, Any]]) -> None:
    """Generates embeddings for document chunks and stores them in memory."""
    texts = [doc["text"] for doc in docs]
    # Generate dense embeddings (returns shape: [N, 384])
    embeddings = self.model.encode(texts, convert_to_numpy=True)

    # Normalize vectors to unit length for fast dot-product similarity
    norms = np.linalg.norm(embeddings, axis=1, keepdims=True)
    normalized_embeddings = embeddings / norms

    self.vectors = np.vstack([self.vectors, normalized_embeddings]) if self.vectors.size > 0 else
normalized_embeddings
    self.documents.extend(docs)
    print(f"Successfully indexed {len(docs)} document chunks.")

  def compute_cosine_similarity(self, query_vector: np.ndarray, doc_vectors: np.ndarray) -> np.ndarray:
    """Computes cosine similarity scores between query vector and document vectors."""
    # For unit-normalized vectors, Cosine Similarity = Dot Product
    return np.dot(doc_vectors, query_vector)

  def search(self, query_text: str, top_k: int = 3) -> List[Dict[str, Any]]:
    """Executes semantic similarity search for a natural language query."""
    # 1. Embed and normalize the query string
    raw_query_vector = self.model.encode([query_text], convert_to_numpy=True)[0]
    query_vector = raw_query_vector / np.linalg.norm(raw_query_vector)

    # 2. Compute similarity scores across all indexed vectors
    similarity_scores = self.compute_cosine_similarity(query_vector, self.vectors)

    # 3. Rank top-k results in descending order
```

```
        top_indices = np.argsort(similarity_scores)[::-1][:top_k]

        results = []
        for idx in top_indices:
            result_item = self.documents[idx].copy()
            result_item["similarity_score"] = float(similarity_scores[idx])
            result_item["cosine_distance"] = float(1.0 - similarity_scores[idx])
            results.append(result_item)

        return results


# --- Execution Walkthrough ---
if __name__ == "__main__":
    # Sample enterprise document chunks
    corpus = [
        {
            "chunk_id": "CHUNK_001",
            "text": "[Doc: Maintenance Manual] Cell temperature spikes above 65C indicate thermal runaway risks in Block B.",
            "equipment": "GIGA-BATT-9000"
        },
        {
            "chunk_id": "CHUNK_002",
            "text": "[Doc: Maintenance Manual] Apply high-grade silicone grease to all primary coolant loop O-rings.",
            "equipment": "GIGA-BATT-9000"
        },
        {
            "chunk_id": "CHUNK_003",
```

```
        "text": "[Doc: ERP Dispatches] Customer Alice Smith logged a billing inquiry regarding invoice INV-4019.",
        "equipment": "N/A"
    },
    {
        "chunk_id": "CHUNK_004",
        "text": "[Doc: Technical SOP] Excessive internal resistance causes rapid voltage drop during high load discharge.",
        "equipment": "GIGA-BATT-9000"
    }
]

# Initialize Engine & Add Corpus
engine = VectorSearchEngine()
engine.add_documents(corpus)

# Execute Semantic Query (Notice: Query uses different words than Chunk 001!)
query = "overheating battery cells and thermal hazards"
print(f"\nUser Query: '{query}'\n" + "-"*50)

searchResults = engine.search(query, top_k=2)
for rank, res in enumerate(searchResults, start=1):
    print(f"Rank {rank} | Score: {res['similarity_score']:.4f} | Distance: {res['cosine_distance']:.4f}")
    print(f"  ID: {res['chunk_id']} | Equipment: {res['equipment']}")
    print(f"  Text: {res['text']}\n")
```

**Review & Preparation Questions**

1. **Embedding Concepts:** Why do dense neural embeddings capture semantic meaning better than sparse keyword representations like TF-IDF or BM25?
2. **Mathematical Mechanics:** Explain why unit-normalizing vectors ($\Vert{}\mathbf{V}\Vert{} = 1$) allows database engines to use Dot Product operations to compute Cosine Similarity efficiently.

3. **Index Trade-offs:** Contrast the execution performance and memory requirements of a brute-force flat search (k-NN) with a graph-based **HNSW** index.
4. **Architectural Choice:** Under what circumstances should an enterprise choose an integrated vector extension (e.g., MariaDB Vector) over a dedicated vector database (e.g., Chroma or Pinecone)?

**References & Further Reading**


- **MariaDB Corporation.** "MariaDB Vector Documentation & Vector Functions." *MariaDB Knowledge Base*, 2026.
- **Malkov, Y. A., & Yashunin, D. A. (2018).** "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- **Reimers, N., & Gurevych, I. (2019).** "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *EMNLP 2019*.
- **Pinecone Systems.** "Vector Indexes Explained: HNSW and IVF." *Pinecone Learning Center*.